

컴퓨터비전 응용 3주차 요약본

12. 강화학습 I(Reinforcement Learning)

12.1. Reinforcement learning (RL)의 개요

- 강화학습이란? (What is Reinforcement Learning?)
 - Trial & Error**(시행착오)와 **Reward**(보상)을 기반으로 학습
 - 에이전트(Agent)가 환경(Environment)과 상호작용하며 최적의 정책(Policy)을 학습

12.2 FrozenLake/Carth-Pole 이해하기

▶ FrozenLake 환경이해

- Environment

Action Space	Discrete(4)
Observation Space	Discrete(16)
import	<code>gymnasium.make("FrozenLake-v1")</code>



Observation Space ¶

The observation is a value representing the player's current position as $\text{current_row} * \text{ncols} + \text{current_col}$ (where both the row and col start at 0).

For example, the goal position in the 4x4 map can be calculated as follows: $3 * 4 + 3 = 15$. The number of possible observations is dependent on the size of the map.

The observation is returned as an `int()`.

- FrozenLake 환경이란?**
 - 얼어붙은 호수 위를 **미끄러지며** 골(goal) 지점까지 이동하는 게임 환경
 - 중간에 구멍(hole)이 있으며, 여기에 빠지면 게임이 종료
- Environment 설정**

구성 요소	설명
Action Space	<code>Discrete(4)</code> — 4가지 방향 이동 가능 (상, 하, 좌, 우)
Observation Space	<code>Discrete(16)</code> — 4x4 격자의 각 칸에 대한 상태 (0~15의 정수)
Import 코드	<code>gymnasium.make("FrozenLake-v1")</code>

→ 강화학습 문제에서 관측 공간(Observation Space)과 행동 공간(Action Space)을 이해하는 것은 필수

→ FrozenLake는 이산적 상태 공간과 이산적 행동 공간을 가지며, 정책 학습을 통해 최적의 이동 경로를 탐색

▶ Cart Pole 환경이해

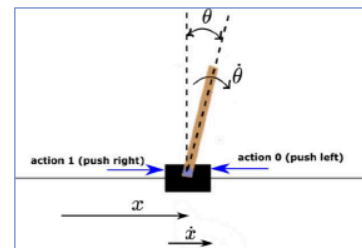
• Environment

Action Space	<code>Discrete(2)</code>
Observation Space	<code>Box([-4.8 -inf -0.41887903 -inf], [4.8 inf 0.41887903 inf], (4,), float32)</code>
import	<code>gymnasium.make("CartPole-v1")</code>

Observation Space ¶

The observation is a `ndarray` with shape `(4,)` with the values corresponding to the following positions and velocities:

Num	Observation	Min	Max
0	Cart Position	-4.8	4.8
1	Cart Velocity	-Inf	Inf
2	Pole Angle	$\sim -0.418 \text{ rad } (-24^\circ)$	$\sim 0.418 \text{ rad } (24^\circ)$
3	Pole Angular Velocity	-Inf	Inf



출처 : https://gymnasium.farama.org/environments/classic_control/cart_pole/

→ 막대(balance beam)를 수레(cart) 위에 세워 균형을 잡는 문제

• Environment 설정

요소	설명
Action Space	Discrete(2) — 가능한 행동은 2개:
0: 왼쪽으로 밀기	
1: 오른쪽으로 밀기	
Observation Space	Box(...) — 연속형 관측 공간. 총 4개의 실수값으로 이루어짐

→ 불연속적인 행동 공간(이산적)과 연속적인 관측 공간(연속적)을 가진 환경입니다.

• Observation Space 상세

- **Cart Position:** 수레의 좌우 위치
- **Cart Velocity:** 수레의 속도
- **Pole Angle (θ):** 막대의 기울기 (라디안 단위, 약 ± 24 도 범위)
- **Pole Angular Velocity:** 막대의 각속도

• 환경 목표

- 수레를 좌우로 이동시켜 막대가 **넘어지지 않고 수직 상태를 유지하도록** 하는 것이 목표
 - Episode는 막대가 일정 각도 이상 기울어지거나, 수레가 경계를 벗어나면 종료
 - 일반적으로 **강화학습 에이전트는 timestep마다 1의 보상**을 받고, 최대한 오래 살아남는 것이 목표

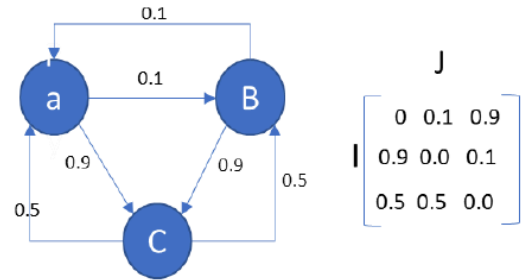
• 요약

- 대표적인 **클래식 강화학습 문제** 중 하나
- 상태는 연속적 (실수값), 행동은 이산적 (좌/우 2가지)
- 최적의 정책은 막대가 넘어지지 않도록 수레를 적절히 조작하는 것

▶ 환경과 모델 (Model)

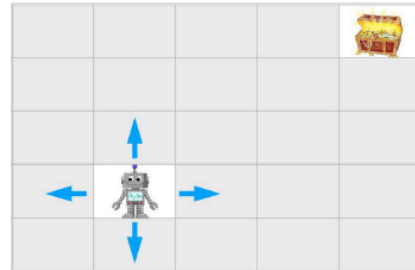
• Model-based

- 각 state에서 상태전이 확률을 알고 있음
- 각 state에서 보상을 알고 있음



• Model-free

- 각 state에서 상태전이 확률을 모르고 있음
- 각 state에서 보상을 모르고 있음



• 오른쪽 상단 그림 <Model-based (모델 기반)>

- 에이전트가 환경에 대한 **이전 지식**을 가지고 있다는 전제 하에 진행
- 상태 a, B, C가 있고, 화살표로 상태 전이 확률이 표시되어 있음
- 예: 상태 a에서 0.1 확률로 B로, 0.9 확률로 C로 이동
- 오른쪽에는 상태 전이 행렬 I와 보상 벡터 J가 주어짐
- 활용
 - **Dynamic Programming** (예: Value Iteration, Policy Iteration) 등의 기법에 사용됨
 - 환경이 "모델링 가능한 경우"에 효과적

• 오른쪽 하단 그림 <Model-free (모델 없음)>

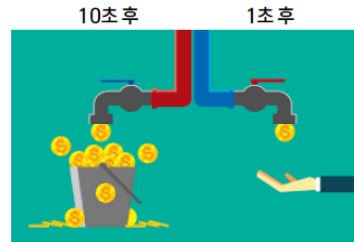
- 에이전트가 **환경 모델을 모르고**, 직접 경험을 통해 학습
 - 실제 환경에서 시행착오를 통해 얻어야 함
- 로봇이 미지의 격자 환경(grid world)에 있으며, 상하좌우로 움직일 수 있음
- 목표 지점은 보물상자 (보상 있음), 하지만 전이 확률과 보상 정보를 모름
- 오직 시행착오를 통해 환경을 탐색하고 학습해야 함
- 활용

- Q-Learning, SARSA, DQN 등
- 복잡하거나 실제 환경에서 환경 모델을 알 수 없을 때 사용

12.3 RL에 필요한 파라미터와 그 역할

▶ 보상 (Reward) vs 가치함수 (Value function)

- 보상 (Reward)
 - 즉각적인 보상
 - r_t : t 시점에서 얻게 되는 보상
- 이득 (Return ~ Gain)
 - Time t 이후의 reward의 총합
 - $G_t = r_{t+1} + \gamma * r_{t+2} + \dots + \gamma^{T-1} * r_T$
- 가치함수 (Value function)
 - 장기적인 관점의 누적보상의 기댓값
 - $E[G_t] = E[r_{t+1} + \gamma * r_{t+2} + \dots + \gamma^{T-1} * r_T]$



승패의 관점



- 보상 (Reward)
 - 예: 바둑 한 수 두고 바로 상대방이 돌을 뺏겼다면, 이때 얻는 보상
- 이득 (Return, Gain)
 - 특정 시점 t 이후의 모든 보상들의 합 (즉, 누적보상)
 - 위 수식에서 γ (감가율, discount factor) $\in (0,1)$
→ 시간이 지날수록 미래 보상은 덜 중요하게 여김
- 가치함수 (Value Function)
 - 현재 상태에서 기대할 수 있는 이득(Return)의 기대값
→ 즉, 미래 보상을 고려한 장기적인 가치

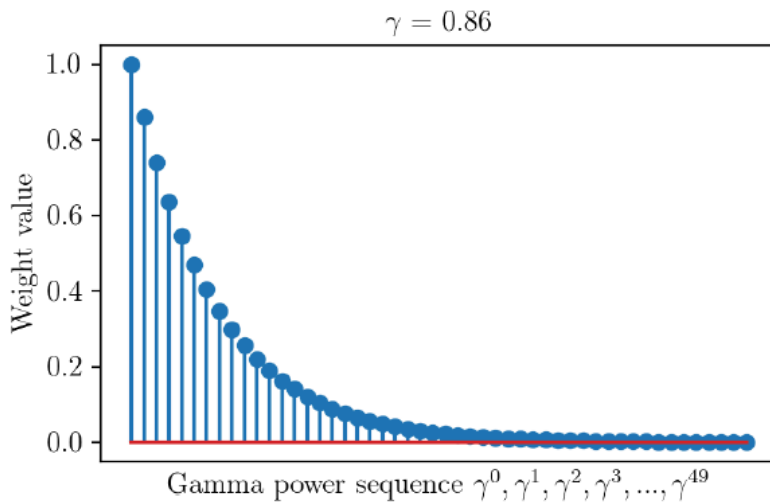
<그림 설명>

- “10초 후 vs 1초 후” 그림
 - 즉각적인 보상보다 미래의 더 큰 보상이 중요할 수 있음
 - 왼쪽 수도꼭지(10초 후): 큰 보상을 늦게 받는 상황

- 오른쪽 수도꼭지(1초 후): 작은 보상을 바로 받는 상황
- 가치 함수는 이 두 보상을 **할인감가(γ)** 해석을 통해 비교하게 됨
- **체스 그림 (로봇)**
 - 즉각적인 한 수는 별 의미 없어 보일 수 있지만,
 - **장기적 시점에서 승리를 위해 전략적인 수를 두는 것이 중요**
 - 이처럼 가치함수는 단기 보상보다 **전체 결과(이득)**를 중요시 함

▶ Discounting factor

$$G_t = r_{t+1} + \gamma * r_{t+2} + \dots + \gamma^{T-1} * r_T$$



할인 계수(Discounting Factor, γ) 의 작동 방식을 시각화 한 그림

- **Discounting Factor (감가율, γ)**
 - 감가율 γ 는 미래 보상의 중요도를 **현재 가치로 환산**할 때 사용하는 계수
 - 보통 $0 < \gamma < 1$ 범위의 값을 가지며, γ 가 클수록 먼 미래의 보상 까지 고려함
- **그래프 해석**

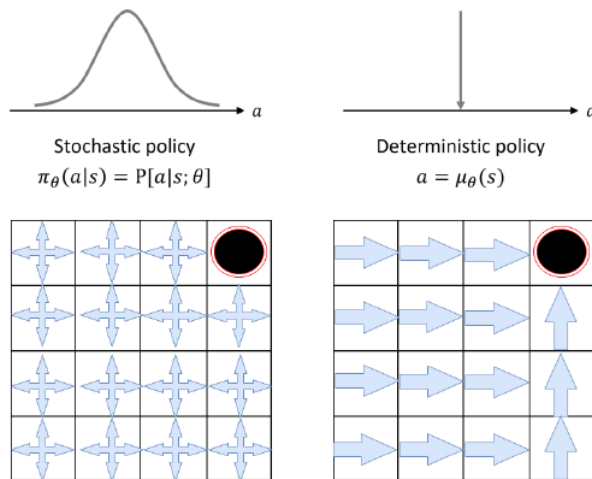
- x축: $\gamma^0, \gamma^1, \gamma^2, \dots, \gamma^{49}$ — 시간 t 이후의 각 시간 스텝에서의 감가율의 거듭제곱
- y축: Weight value — 실제로 보상에 곱해지는 가중치 값
- 여기서는 $\gamma = 0.86$ 로 설정되어 있고,
 - $\gamma^0 = 1.0$
 - $\gamma^1 = 0.86$
 - $\gamma^2 = 0.7396$
 - ...
 - γ 가 거듭제곱될수록 감가 효과가 커지고, 미래의 보상은 더 작게 반영됨

• 정리 요약

- 감가율(γ)은 강화학습에서 **미래 보상의 현재 가치 반영 정도**를 조절함
- γ 값이 클수록 먼 미래의 보상도 충분히 고려함 → 장기적 전략
- γ 값이 작을수록 현재 보상에 집중함 → 단기적 행동 유도

▶ 정책 (Policy)

- 결정론적 (Deterministic)
 - $\pi(a_t|s_t) = k$
- 확률론적 (Stochastic)
 - $\pi(a_t|s_t) \sim P(a_t|s_t, \theta)$



• 정책(Policy)이란?

→ 정책이란 에이전트(agent)가 각 상태(state)에서 어떤 행동(action)을 선택할 지 결정하는 전략

• 정책의 유형

1. 결정론적 정책 (Deterministic Policy)

- 정의: 주어진 상태에서 항상 같은 행동을 선택
- 수식: $\pi(a_t|s_t) = k$, 또는 $a = \mu_\theta(s)$
- 예: 상태 s_1 에서 항상 오른쪽으로 이동

오른쪽 그림 설명 (Deterministic policy):

- 각 상태에서 하나의 고정된 방향으로만 이동
- 정책이 딱 한 가지 행동만 선택

2. 확률론적 정책 (Stochastic Policy)

- 정의: 주어진 상태에서 확률에 따라 여러 행동 중 하나를 선택
- 수식: $\pi(a_t|s_t) \sim P(a_t|s_t, \theta)$
- 예: 상태 s_1 에서 70% 확률로 오른쪽, 30%로 아래로 이동

왼쪽 그림 설명 (Stochastic policy):

- 각 상태에서 여러 방향의 화살표가 있고, 굵기가 확률을 의미
- 정책이 확률 분포에 따라 행동 선택
- **Deterministic policy** → 탐색이 적고 빠른 실행에 적합
- **Stochastic policy** → 탐색을 통해 더 나은 정책을 찾을 가능성이 높음 (ex. PPO, A3C 등에서 사용)

12.4 Markov decision process (MDP) 이해

▶ Markov decision process (MDP)

- Action A_t



$$MP(S, A, P, R, \gamma)$$

- Policy $\pi(a|s) = P(A_t = a|S_t = s)$

- Action-value function $Q_\pi(s, a) = \mathbb{E}_\pi[G_t|S_t = s, A_t = a]$

$\langle S, A, P_{tran}, R, \gamma \rangle$

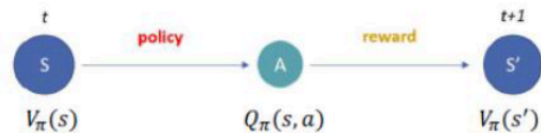
S : set of states

A : set of actions

P_{tran} : state transition probability matrix

R : Reward function

γ : Discounting factor



• Markov Decision Process (MDP)의 구성 요소

MDP는 마르코프 과정(MP)에 행동(Action)과 보상(Reward) 요소를 추가한 구조

MDP 구성:

$$MDP = \langle S, A, P, R, \gamma \rangle$$

기호	설명
S	상태(state) 공간: 환경의 가능한 모든 상태들
A	행동(action) 공간: 에이전트가 취할 수 있는 모든 행동들
P 또는 $(P_{s'})$	s, a)
R	보상 함수: 어떤 상태에서 특정 행동을 했을 때 받는 보상
γ	할인율(Discount Factor): 미래 보상의 현재 가치 반영 정도 (0~1)

• 동작의 흐름

1. 시간 t 시점에서 에이전트는 상태 $S_t = s$ 에 있음.
2. 정책(policy) π 에 따라 행동 $A_t = a$ 를 선택함.
3. 환경은 이에 반응하여:
 - 보상 $r \sim R(s, a)$ 를 주고,
 - 다음 상태 $S_{t+1} = s'$ 로 전이됨.

→ 이 과정이 반복되며, 에이전트는 최대 보상을 얻기 위해 학습합니다.

• 정책

에이전트의 행동 전략을 의미합니다.

- 확률적 정책: $\pi(a|s) = P(A_t = a \mid S_t = s)$
- 결정론적 정책: $a = \mu(s)$

• 가치 함수(Value Function)

◦ 상태 가치 함수

정책 π 하에서, 상태 s 부터 시작하여 받을 기댓 보상 총합.

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

◦ 행동 가치 함수

상태 s 에서 행동 a 를 했을 때, 그 후 정책 π 를 따를 때 받을 기댓 보상 총합.

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$$

→ 그림에서 **빨간 박스**로 강조된 수식이 이 내용

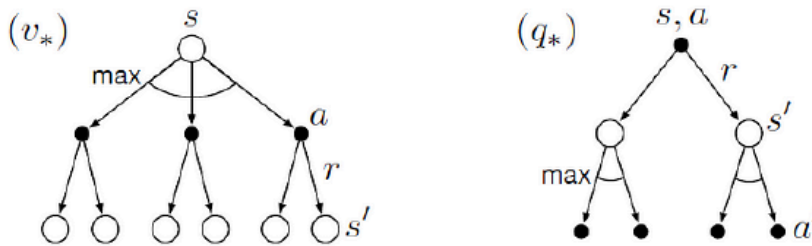
• 관계 요약

이미지 하단 오른쪽 흐름 설명:

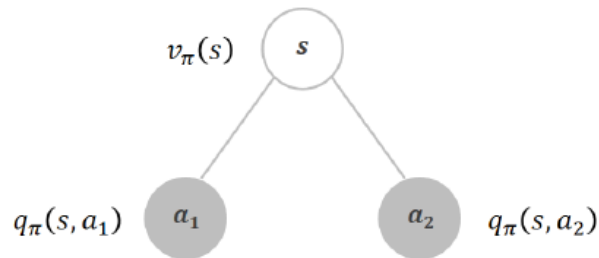
- $V_{\pi}(s) \rightarrow$ 상태에서 얻을 기대 보상
- 정책(policy)에 따라 행동 a 선택 $\rightarrow Q_{\pi}(s, a)$
- 행동 후 상태 전이 $\rightarrow s'$
- 다음 상태에서 $V_{\pi}(s')$ 로 이어짐

• Backup Diagram

► Backup diagram



Backup diagrams for v_* and q_*



$$v_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) q_{\pi}(s, a)$$

"Backup"이란 미래 보상을 현재 가치에 반영(업데이트)하는 연산을 의미하며, 벨만 방정식을 구현하는 시각적 표현

상단 왼쪽: $v_*(s)$ 에 대한 Backup (최적 상태 가치 함수)

- 현재 상태 s 에서 가능한 모든 행동 a 에 대해
- 각 행동 후 도달할 수 있는 다음 상태 s' 들에서
- 기대 보상을 계산한 후 최댓값을 취함.

$$v_*(s) = \max_a \mathbb{E}[r + \gamma v_*(s') \mid s, a]$$

- 의미: 최적의 정책을 따랐을 때, 상태 s 의 가치는 다음 상태들 중 가장 좋은 선택으로 계산됨.

상단 오른쪽: $q_*(s, a)$ 에 대한 Backup (최적 행동 가치 함수)

- 특정 상태 s 와 행동 a 에서
- 다음 상태 s' 로 전이되고,
- 그 이후에는 가능한 행동 a' 중 최적의 행동을 선택한다고 가정.

$$q_*(s, a) = \mathbb{E}[r + \gamma \max_{a'} q_*(s', a')]$$

- 의미: 하나의 상태-행동 쌍에 대해 미래의 보상을 고려한 업데이트.

12.5 가치함수

▶ Bellman Expectation Equation : 상태 가치함수 ($V(s)$, State-value function)

The **value function** can be decomposed into two parts:

- immediate reward R_{t+1}
- discounted value of successor state $\gamma V(S_{t+1})$

$$\begin{aligned} v(s) &= \mathbb{E}[G_t \mid S_t = s] \\ &= \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s] \\ &= \mathbb{E}[R_{t+1} + \gamma (R_{t+2} + \gamma R_{t+3} + \dots) \mid S_t = s] \\ &= \mathbb{E}[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\ &= \mathbb{E}[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s] \end{aligned}$$

→ 강화학습에서 매우 핵심적인 수식인 벨만 기대 방정식을 상태 가치 함수 $V(s)$ 관점에서 풀어 설명한 것

• Bellman Expectation Equation

특정 정책을 따를 때 **상태의 가치를 즉각적인 보상 + 다음 상태의 할인된 가치로 분해 가능**을 보여주는 식

- 식 구조 해석

$$v(s) = \mathbb{E}[G_t \mid S_t = s]$$

- 상태 s 에서 시작했을 때, 미래의 총 보상 G_t 의 기댓값.
- 이를 시간축으로 펼치면 아래와 같이 무한합의 구조로 표현됩니다:

$$= \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \mid S_t = s]$$

- 첫 번째 항: 즉각적인 보상 R_{t+1}
- 나머지 항: 다음 상태에서 기대되는 총 보상 G_{t+1} , 이를 다시 $v(S_{t+1})$ 로 표현 가능

- 최종 정리된 수식

$$v(s) = \mathbb{E}[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s]$$

- 현재 상태에서의 가치는:
 - 지금 받는 보상 R_{t+1} +
 - 다음 상태의 가치 $v(S_{t+1})$ 를 감가(discount factor γ)하여 더한 것

- 초록색 밑줄 친 부분의 의미

Immediate reward R_{t+1}	지금 상태에서 행동을 했을 때 얻는 보상
Discounted value $\gamma v(S_{t+1})$	다음 상태에서의 장기적 보상을 감가해서 더한 값

- 상태가치 함수

▶ 상태가치 함수 (State value function) 와 policy

$$\begin{aligned}
 v_{\pi}(s) &\doteq \mathbb{E}_{\pi}[G_t \mid S_t = s] && \text{MC method} \\
 &= \mathbb{E}_{\pi}[R_{t+1} + \gamma G_{t+1} \mid S_t = s] && \text{TD method} \\
 &= \sum_a \pi(a|s) \sum_{s'} \sum_r p(s', r|s, a) \left[r + \gamma \mathbb{E}_{\pi}[G_{t+1} | S_{t+1} = s'] \right] \\
 &= \sum_a \pi(a|s) \left[\sum_{s', r} p(s', r|s, a) \left[r + \gamma v_{\pi}(s') \right] \right], \quad \text{for all } s \in \mathcal{S},
 \end{aligned}$$

$$\begin{aligned}
 Q_{\pi}(s, a) &= q_{\pi}(s, a) = \\
 &\sum_{s', r} p(s', r|s, a) [r + \gamma V_{\pi}(s')]
 \end{aligned}$$

벨만 최적 방정식

$$v_*(s) = \max_{\pi} v_{\pi}(s)$$

$$q_*(s, a) = \max_{\pi} q_{\pi}(s, a)$$

◦ 상태 가치 함수 전개 – MC 방식 vs TD 방식

▪ MC 방식 (Monte Carlo)

$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma \mathbb{E}_{\pi}[G_{t+1} \mid S_{t+1} = s']]$$

- 전체 에피소드 종료까지 G_t 를 구해 기대값을 계산
- 정확하지만 샘플 수 많아야 함, 느림

▪ TD 방식 (Temporal Difference)

$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma v_{\pi}(s')]$$

- 다음 상태의 **예상 가치 함수 $v_{\pi}(s')$ **로 근사
- 빠르고 효율적

◦ Action-value 함수 (Q 함수)

$$Q_{\pi}(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma V_{\pi}(s')]$$

- 특정 상태 s 에서 행동 a 를 선택했을 때의 기대 가치
- 상태 가치 $V_{\pi}(s)$ 보다 더 세밀한 평가 가능

12.6 Bellman 방정식과 Dynamic programming

► Dynamic programming

- Policy evaluation (정책평가)

- 정책평가 – state value update

- Policy iteration (정책반복)

- 정책평가 (evaluation)

- 정책향상 (improvement) e.g., Optimal policy

$$\begin{aligned} v_{\pi}(s) &\doteq \mathbb{E}_{\pi}[G_t \mid S_t = s] \\ &= \mathbb{E}_{\pi}[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\ &= \sum_a \pi(a|s) \sum_{s'} \sum_r p(s', r | s, a) [r + \gamma \mathbb{E}_{\pi}[G_{t+1} | S_{t+1} = s']] \\ &= \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')], \quad \text{for all } s \in \mathcal{S}, \end{aligned}$$

- Value iteration (가치 반복)

- 정책평가 + 정책향상을 동시

1. Policy Evaluation (정책 평가)

- 정의: 주어진 정책 π 에 따라 각 상태 s 의 상태 가치 함수 $v_{\pi}(s)$ 를 계산
- 수식 기반:

$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')]$$

- 목적: 정책이 고정되었을 때, 이 정책이 얼마나 좋은지 평가 (state value update)

2. Policy Iteration (정책 반복)

- 두 단계의 반복:

1. 정책 평가 (evaluation): 주어진 정책의 상태 가치 함수 계산

2. 정책 향상 (improvement): 각 상태에서 더 좋은 행동 선택 → 새로운 정책 생성

- 반복하면서 최적 정책에 수렴
- 일반적으로 수렴 속도가 빠름

3. Value Iteration (가치 반복)

- 정책 평가 + 정책 향상을 하나로 결합
- 상태 가치 함수만 업데이트하면서 간접적으로 정책을 향상시킴:

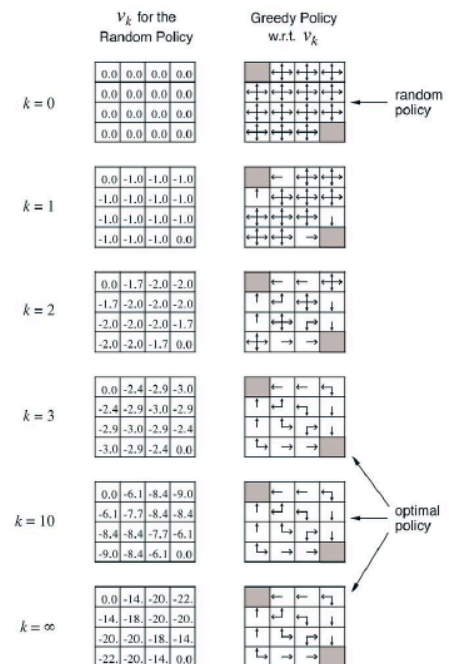
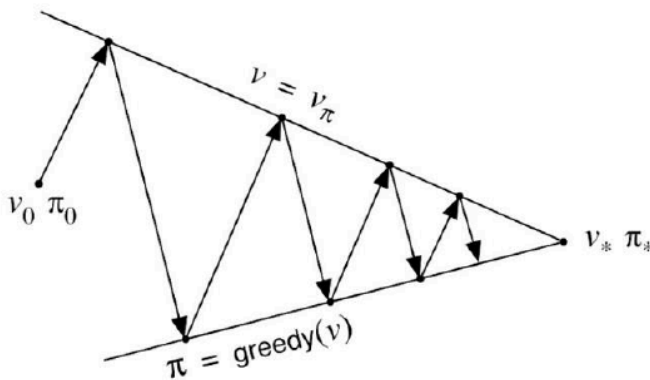
$$v_{k+1}(s) = \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma v_k(s')]$$

- 계산량이 줄어드는 장점

▶ 정책반복의 수렴

(Iterative policy iteration convergence)

- the environment is a finite Markov Decision Process (finite MDP)



정책 반복(Policy Iteration) 기법이 어떻게 수렴하여 최적 정책을 찾는지를 시각적으로 설명합니다. 이는 강화학습에서 대표적인 **Dynamic Programming** 기법 중 하나이며, 유한 상태 공간(finite MDP)을 가정

- 정책 반복의 수렴 과정 (Iterative Policy Iteration Convergence)

[좌측 삼각도식]

- v_0, π_0 : 초기 정책과 가치 함수
- 위로 상승하는 화살표: 현재 정책에 대한 정책 평가 단계 → 새로운 v_k 를 계산
- 아래로 향하는 화살표: 현재 가치 함수로부터 정책 향상 (greedy policy) 단계 → 새로운 π_k 도출
- 이 과정을 반복하며 점점 더 좋은 정책으로 수렴

[우측 표]

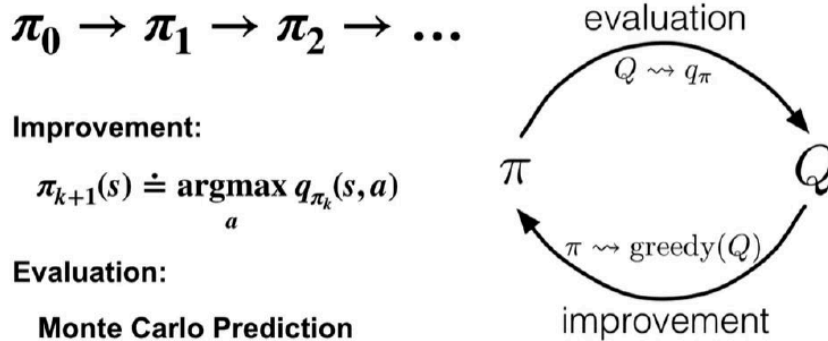
- $k = 0$: 랜덤 정책 기반의 초기 가치 함수 $\rightarrow$ 모두 0
- $k = 1, 2, 3, \dots, 10$: 점차 업데이트되는 가치 함수와 그에 따른 greedy 정책
- $k = \infty$: 최적 정책에 수렴함을 확인 가능
 - 가장 오른쪽에는 각 상태에서의 최적 행동($\rightarrow \downarrow \leftarrow \uparrow$ 등)이 시각화됨
- 수렴 조건
 - 환경이 유한 MDP일 경우, 정책 반복은 항상 최적 정책에 수렴
 - 계산량이 적지는 않지만, 정확한 정책을 얻을 수 있음
- 정리

단계	설명
정책 평가	현재 정책에 대해 $v_{\pi}(s)$ 계산
정책 향상	v_{π} 를 바탕으로 $\pi \leftarrow \text{greedy}(v_{\pi})$
수렴	$\pi_k = \pi_{k+1}$ 이면 수렴 완료

13. 강화 학습 II (Reinforcement Learning)

13.1 Monte-Carlo (MC) method의 작동 원리

Monte Carlo Generalized Policy Iteration



강화 학습에서 정책과 가치 함수를 번갈아 가며 개선하는 정책 반복(Policy Iteration)의 일반적인 구조를 **Monte Carlo 방식**으로 구현한 개념

- **Monte Carlo Generalized Policy Iteration (MC-GPI)**

- 정책 평가 (Policy Evaluation)와 정책 개선 (Policy Improvement)을 번갈아 수행
- 평가 방법으로 **Monte Carlo Prediction** 사용
- 개선은 **greedy policy** (현재 가치 함수에 대해 가장 좋은 행동을 택하는 정책)

- 구성 요소

1. Policy Evaluation

- 현재 정책 π_k 를 고정한 채로, **MC Prediction**을 통해 상태-행동 가치 함수 $q_{\pi_k}(s, a)$ 를 추정
- MC 방식은 완전한 에피소드 종료 후 얻은 return 값을 기반으로 기대값을 계산함

2. Policy Improvement

- 다음 정책 π_{k+1} 은 다음과 같이 정의됨:

$$\pi_{k+1}(s) \doteq \operatorname{argmax}_a q_{\pi_k}(s, a)$$

- 즉, π_k 의 Q 함수 기반으로 가장 높은 값을 주는 행동 a 를 선택하여 새 정책 구성

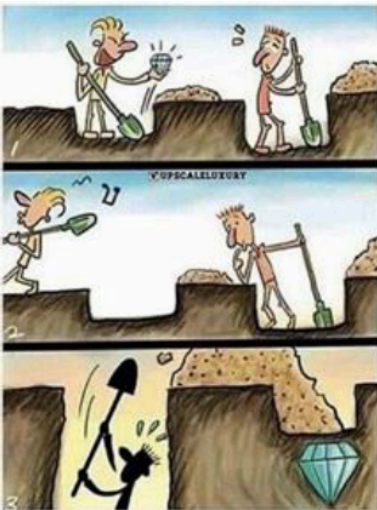
- 반복 구조

- $\pi \rightarrow q_\pi \rightarrow \operatorname{greedy}(q_\pi) \rightarrow \pi \rightarrow \dots$
- 현재 정책으로 행동하며 MC 방식으로 가치 추정
- 이를 기반으로 더 나은 정책을 도출

- 다시 이 정책으로 행동하며 평가 반복
- **장점**
 - 환경 모델 없이도 작동 (model-free)
 - Monte Carlo 방식으로 비교적 간단하게 구현 가능
- **단점**
 - 에피소드 단위로만 학습 가능 (On Policy)
 - 수렴 속도가 느릴 수 있음

13.2 e-greedy policy

Exploration & exploit (decaying ϵ - greedy)



$$A_t \leftarrow \begin{cases} \operatorname{argmax} Q_t(a) & \text{with probability } 1 - \epsilon \\ a \sim \operatorname{Uniform}(\{a_1 \dots a_k\}) & \text{with probability } \epsilon \end{cases}$$

• ϵ -Greedy Policy란?

에이전트는 다음 중 하나를 선택합니다:

$$A_t \leftarrow \begin{cases} \operatorname{arg max} Q_t(a) & \text{with probability } 1 - \epsilon \quad (\text{Exploit}) \\ a \sim \operatorname{Uniform}(\{a_1, \dots, a_k\}) & \text{with probability } \epsilon \quad (\text{Explore}) \end{cases}$$

- **Exploit:** 현재까지의 경험 중 가장 보상이 큰 행동을 선택
- **Explore:** 무작위 행동을 통해 새로운 전략을 탐색

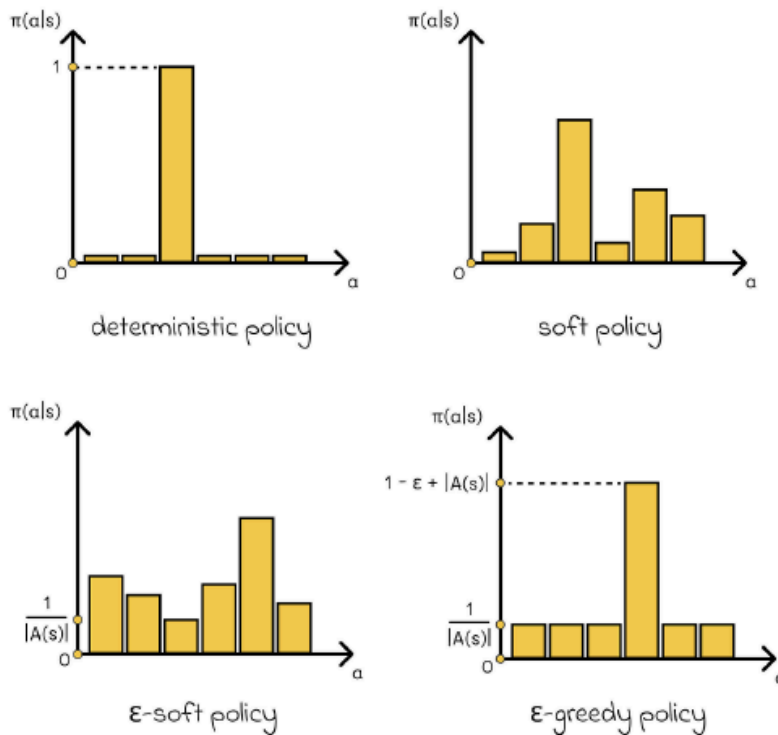
- **Decaying ϵ -Greedy**

- 학습 초반엔 ϵ 값을 높게 설정 → 다양한 시도
- 학습이 진행되면서 ϵ 를 점차 줄임 → 안정적인 exploit 위주 정책

- 정리

개념	설명
Exploit	현재의 최적 행동을 선택
Explore	무작위 행동으로 새로운 정보 획득
ϵ -Greedy	두 방법의 확률적 혼합
Decaying ϵ	학습이 진행될수록 탐험을 줄임

▶ **e-soft policy vs e-greedy policy**



- **Deterministic Policy (결정론적 정책)**

- 특정 상태 s 에서 **하나의 행동** a 만을 항상 선택함
- 확률 분포: $\pi(a | s) = 1$ for some a , 그 외는 0
- 장점: 실행 단순
- 단점: **탐험(Exploration)이 없음** → 지역 최적에 빠질 위험 있음
- **Soft Policy**
 - 여러 행동을 **확률적으로 선택**
 - 어떤 행동도 확률 0이 아님
 - 예: Softmax policy 등
 - 탐험과 활용 간의 **자연스러운 균형 유지**
- **ϵ -soft Policy**
 - 모든 행동에 일정 확률(> 0)을 부여
 - 특정 행동을 선호하되, **모든 행동이 선택될 가능성 존재**
 - 보장된 탐험성 → 정책이 수렴해도 **탐험을 완전히 잃지 않음**
- **ϵ -greedy Policy**
 - 대부분의 경우 **가장 좋은 행동(greedy)** 선택
 - 나머지는 **무작위로 선택**
 - ϵ 값이 작을수록 **활용 위주**, 클수록 **탐험 위주**
- **정리**

Deterministic	한 행동만 선택	봉 하나만 높음
Soft	여러 행동에 다른 확률	다양한 높이
ϵ -soft	균일 분포에 가까운 확률	고르게 분포됨
ϵ -greedy	하나는 높고 나머진 작음	가장 큰 봉 하나 + 작은 봉 여러 개

→ **ϵ -greedy**는 탐험이 제한적이며 특정 행동에 집중

→ **ϵ -soft**는 모든 행동을 일정 확률 이상 선택하게 하여 더 **균형 있는 탐험** 제공

13.3 Q-learning

Q-learning은 **모델 없이(optimal policy without knowing the environment)** 최적의 행동 전략(policy)을 학습하는 대표적인 강화학습 알고리즘

Q-learning algorithm

For each s, a initialize table entry $\hat{Q}(s, a) \leftarrow 0$

Observe current state s

Do forever:

- Select an action a and execute it
- Receive immediate reward r
- Observe the new state s'
- Update the table entry for $\hat{Q}(s, a)$ as follows:
$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a')]$$
- $s \leftarrow s'$

Machine Learning, T. Mitchell, McGraw Hill, 1997

► Q-learning

$$Q(s, a) = r(s, a) + \max_a Q(s', a)$$

- **Off-policy, model-free** 강화학습 알고리즘
- 목적: 최적의 행동 가치 함수 $Q^*(s, a)$ 를 학습하여 최적 정책을 도출
- 미래 보상을 최대화하는 행동을 선택

→ 현재 Q값을 업데이트할 때, 실제로 받은 보상과 앞으로 받을 수 있는 최대 보상을 고려하여 조금씩 보정

- **Q-learning 업데이트 수식**

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)$$

이 이미지에서 단순화된 수식:

$$Q(s, a) = r(s, a) + \max_{a'} Q(s', a')$$

- 각 요소 설명

항목	의미
s	현재 상태 (state)
a	현재 상태에서 취한 행동 (action)
$r(s, a)$	즉시 보상 (reward)
s'	행동 후 도달한 다음 상태 (next state)
$\max_{a'} Q(s', a')$	다음 상태에서 가능한 행동 중 최대 Q값 (미래 보상의 추정)
$Q(s, a)$	상태-행동 쌍의 기대되는 누적 보상 (현재 가치 추정)
α	학습률
γ	할인율 (미래 보상에 대한 중요도)

- 알고리즘 단계별 설명

1. 초기화 (Initialization)

모든 상태-행동 쌍 $Q(s, a)$ 에 대해 값을 0으로 초기화:

$$\hat{Q}(s, a) \leftarrow 0$$

2. 초기 상태 관찰

- 현재 상태 s 를 관찰합니다.

3. 반복 루프 (Do forever)

- 행동 선택: 현재 정책에 따라 행동 a 를 선택하고 실행
- 보상 수신: 환경으로부터 즉각적인 보상 r 을 받음
- 다음 상태 관찰: 새로운 상태 s' 를 관찰
- Q값 업데이트 (핵심)

$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') \right]$$

- α : 학습률 (learning rate)
- γ : 할인율 (discount factor)
- $\max_{a'} Q(s', a')$: 다음 상태에서의 최적 행동 가치
- 상태 이동:

$$s \leftarrow s'$$

항목	의미
$Q(s, a)$	상태 s 에서 행동 a 를 취했을 때 기대되는 보상의 총합
α	학습률 - 새 정보에 대한 반영 정도
γ	할인율 - 미래 보상의 현재 가치 반영
$\max Q(s', a')$	다음 상태에서 가능한 행동 중 가장 큰 Q값 선택 (탐욕적 접근)

• 정리

- Q-learning은 **off-policy** 방식
(탐험 정책과 업데이트 정책이 다름 - 탐색은 ϵ -greedy, 업데이트는 greedy)
- 환경의 모델을 몰라도 최적 정책을 학습할 수 있음

- 실시간으로 상태를 관찰하고 보상을 받으며 **Q값을 반복적으로 업데이트**
- **Q-learning의 discount factor γ**

▶ Q – learning with discount factor γ

$$Q(s, a) = r(s, a) + \gamma \max_a Q(s', a)$$

Diagram illustrating the Q-learning equation with labels:

- $Q(s, a)$: state (pointing to s) and action (pointing to a)
- $r(s, a)$: reward (pointing to r)
- γ : discount factor (pointing to γ)
- $\max_a$: action (pointing to a)
- $Q(s', a)$: state (pointing to s') and action (pointing to a)

구성 요소	의미
$Q(s, a)$	상태 s 에서 행동 a 를 했을 때의 Q-value (미래 보상의 추정치)
$r(s, a)$	즉각적인 보상 (reward)
γ	Discount factor (할인율) — 미래 보상의 현재 가치 반영
s'	다음 상태 (state transition 이후 도달하는 상태)
$\max_{a'} Q(s', a')$	다음 상태에서 가능한 행동 중 가장 큰 Q값 (미래 최적 행동의 가치)

• 할인율 γ 의 역할

- $\gamma \in [0, 1]$
- 미래 보상에 부여하는 중요도의 척도:
 - $\gamma = 0$: 오직 즉시 보상만 고려 (myopic)
 - $\gamma \rightarrow 1$: 장기적인 보상까지 고려 (far-sighted)

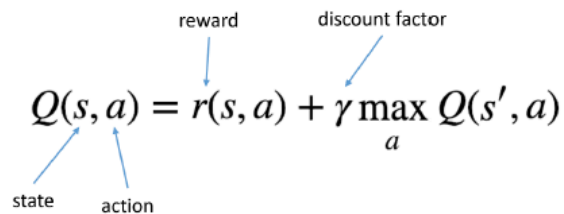
즉, γ 는 “얼마나 멀리까지 미래의 보상을 신뢰할 것인가”를 결정

• TD Learning

▶ Q-learning in stochastic (real) world

- TD learning

α = learning rate

$$Q(s, a) = r(s, a) + \gamma \max_a Q(s', a)$$


$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a')]$$

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

- TD Learning의 특징

- 온-라인 학습 가능: 에피소드 전체를 기다릴 필요 없이 한 번의 transition으로 업데이트 가능
- 샘플 기반 근사: 확률적으로 reward와 transition이 바뀌는 현실에서도 견고하게 동작
- 점진적 업데이트: 기존 Q값을 유지하면서 천천히 바뀌어 나감

- 수식의 사용 분야

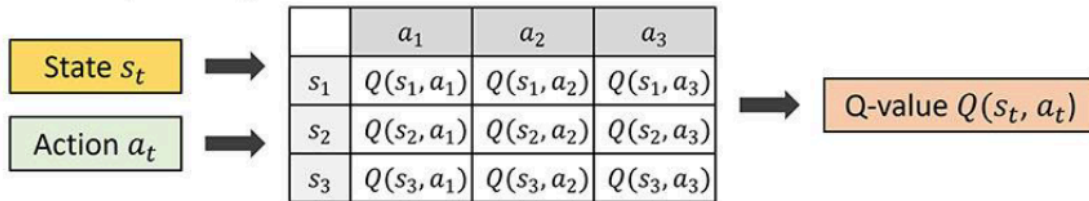
- 현실적 환경(예: 게임, 로봇 등)에서는 보상이 일정하지 않기 때문에, **Q값을 확률적으로 업데이트**
- 학습률 α 는 지나치게 큰 변화 방지 역할
- 이는 TD 학습의 핵심 공식

14. 강화 학습 III (Reinforcement Learning)

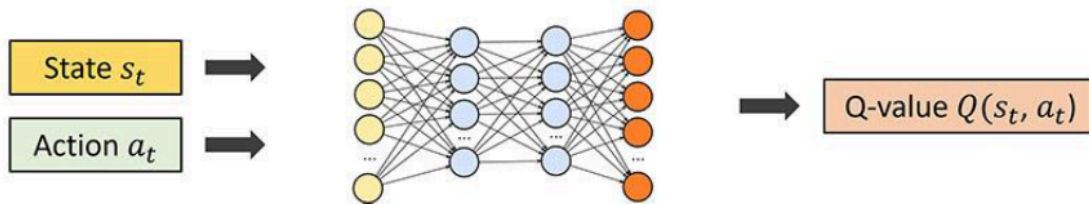
14.1 Q-learning과 Deep Q-learning(DQN)의 차이

▶ Q - DNN

Classic Q-learning



Deep Q-learning



• Classic Q-learning (위쪽 그림)

구성 요소:

- 상태 s_t 와 행동 a_t 에 대응되는 **Q-value** $Q(s_t, a_t)$ 를 테이블로 저장
- Q-table은 가능한 모든 상태와 행동 쌍에 대해 값을 저장하는 방식

특징:

- 테이블 기반 방식
- 상태나 행동의 수가 작을 경우에만 효율적
- 상태공간이 커질수록 메모리 및 연산량 폭발

• Deep Q-learning (아래쪽 그림)

구성 요소:

- 상태 s_t 를 입력으로 받고, **Q-value** $Q(s_t, a_t)$ 를 출력하는 신경망(DNN) 사용
- 네트워크는 각 행동에 대한 Q값을 동시에 출력

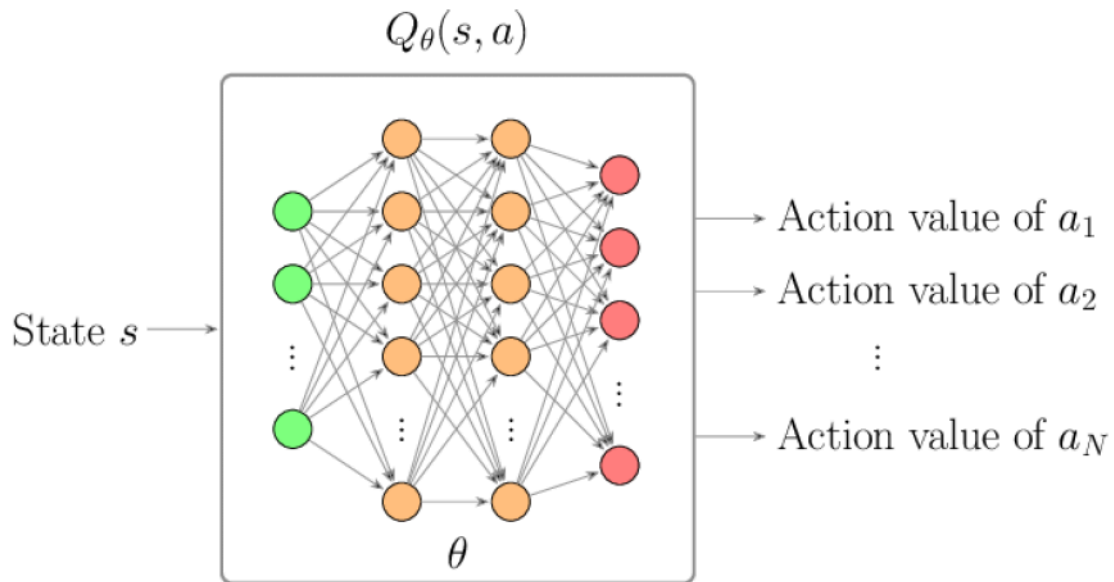
특징:

- **Function Approximation** 기반
- 테이블 대신 신경망이 **Q-value**를 근사
- 고차원 또는 연속적인 상태 공간에 적합
- DQN의 핵심 구성 요소:
 - Experience Replay (경험 재생)
 - Target Network (타겟 네트워크)

• 정리

항목	Classic Q-learning	Deep Q-learning (DQN)
상태 표현	이산(discrete)	연속 or 고차원 가능
Q-value 저장 방식	Q-table	Neural Network (DNN)
확장성	낮음 (작은 상태공간만 가능)	높음 (복잡한 환경에도 적합)
일반화 능력	없음	있음

▶ The value-based DQN



• 전체 구조

$Q_{\theta}(s, a)$: 주어진 상태 s 에 대해 가능한 모든 행동 $a_1, a_2, \dots, a_N$ 의 Q값을 한 번의 **forward pass**로 예측

신경망의 파라미터 θ 를 통해 학습

• 구성 요소

1. 입력층 (왼쪽 초록색 노드들)

- 입력: 상태 벡터 s
- 예: 이미지, 센서값, 게임 상태 등

2. 은닉층 (중간 오렌지색 노드들)

- Fully connected layer 또는 Conv layer 사용 가능
- 입력 상태를 고차원 특징으로 인코딩

3. 출력층 (오른쪽 붉은색 노드들)

- 출력: 상태 s 에 대한 모든 행동 $a_1, a_2, \dots, a_N$ 의 Q-value
- 예: CartPole에서는 2개(왼쪽, 오른쪽), Atari에서는 4~18개 등

- **DQN의 핵심 테크닉**

1. **Target Network:**

- 안정적인 학습을 위해 θ^- 는 일정 주기로만 업데이트

2. **Experience Replay:**

- (s, a, r, s') 튜플을 저장하고 무작위로 샘플링하여 학습
- 샘플 간 상관관계를 줄여 학습 효율 개선

3. **ϵ -greedy Policy:**

- 탐험(Explore)과 활용(Exploit)을 병행

Convergence

$\hat{Q}$ denote learner's current approximation to Q .

$$\min_{\theta} \sum_{t=0}^T [\hat{Q}(s_t, a_t | \theta) - (r_t + \gamma \max_{a'} \hat{Q}(s_{t+1}, a' | \theta))]^2$$

- ▶ Converges to Q^* using table lookup representation
- ▶ But **diverges** using neural networks due to:
 - ▶ Correlations between samples
 - ▶ Non-stationary targets

Tutorial: Deep Reinforcement Learning, David Silver, Google DeepMind

- $\hat{Q}(s_t, a_t | \theta)$: 현재 Q-network가 예측한 Q값
- $r_t + \gamma \max_{a'} \hat{Q}(s_{t+1}, a' | \theta)$: 타겟 Q값 (현재 보상 + 다음 상태의 최대 Q값)
- 목적: 예측값과 타겟값의 오차를 최소화

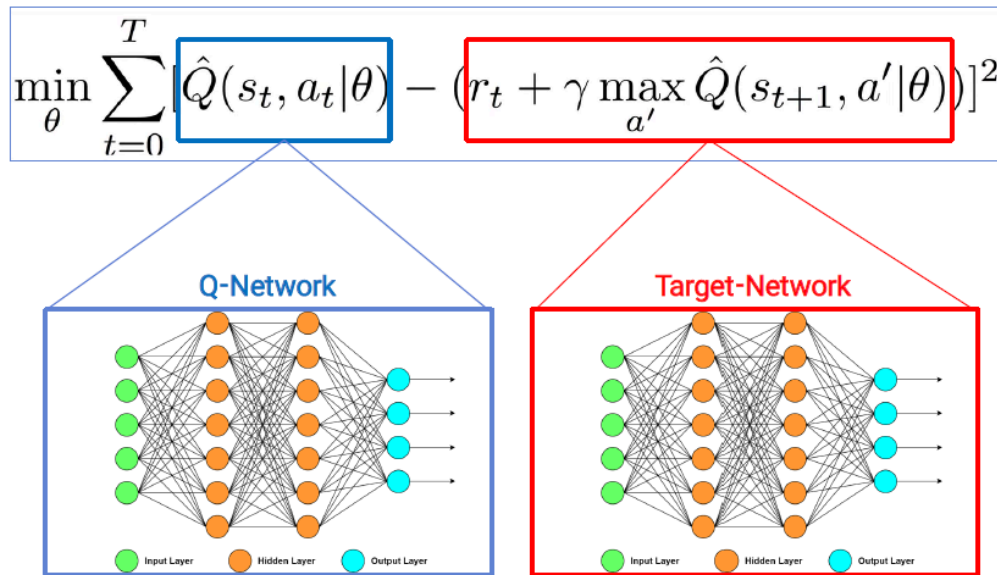
- **수렴(Convergence)**

- 테이블 방식일 때 (Tabular Q-learning)

- 각 $Q(s, a)$ 값이 명시적으로 저장됨

- 모든 state-action 쌍을 충분히 방문하면 Q^* (최적 Q 함수)에 수렴함
- 증명된 안정적인 수렴 보장 있음
- **신경망 방식일 때 (Neural Network 기반 DQN)**
 - Q함수를 근사하기 위해 파라미터 θ 를 갖는 **신경망** 사용
 - 이 경우는 **발산(Divergence)** 가능성 있음
- **발산의 원인**
 - **Samples 간 상관관계 (Correlations between samples)**
 - 연속적인 샘플들은 매우 유사하며, 이는 SGD의 기본 가정(독립적인 샘플)을 위반함
 - 해결책: **Experience Replay**
 - **타깃 값의 변화 (Non-stationary targets)**
 - 타깃 $r + \gamma \max Q(s', a')$ 계산도 신경망에 의존 $\rightarrow$ 학습 중 타깃도 계속 바뀜
 - 해결책: **Target Network** 도입 (주기적으로 복사해서 고정함)
- **정리**
 - **Q-learning**은 테이블 방식으로는 수렴하지만, 딥러닝 기반으로 구현하면 **발산**할 수 있음
 - 이는 **샘플 간 상관관계**와 **타깃의 비정상성** 때문
 - 안정적 학습을 위해 **Experience Replay, Target Network** 등의テクニック이 필요함

▶ DQN architecture intuition



• DQN에서의 타겟 분리 학습 구조

◦ Q-Network vs Target-Network

Q-Network	Target-Network
$\hat{Q}(s_t, a_t \theta)$	$\hat{Q}(s_{t+1}, a' \theta^-)$
학습 중 파라미터가 계속 업데이트됨	일정 주기마다 Q-Network의 파라미터로 복사
현재 상태에 대해 예측값을 제공	타겟값을 안정적으로 생성

→ **Q-Network**는 수식의 왼쪽 항 (예측값)을 생성하는 네트워크

→ **Target-Network**는 수식의 오른쪽 항 (타겟값)을 생성

→ 이렇게 두 네트워크를 분리함으로써, Q-learning의 **발산 문제**를 줄이고 학습의 안정성을 확보

◦ 왜 Target-Network가 필요한가?

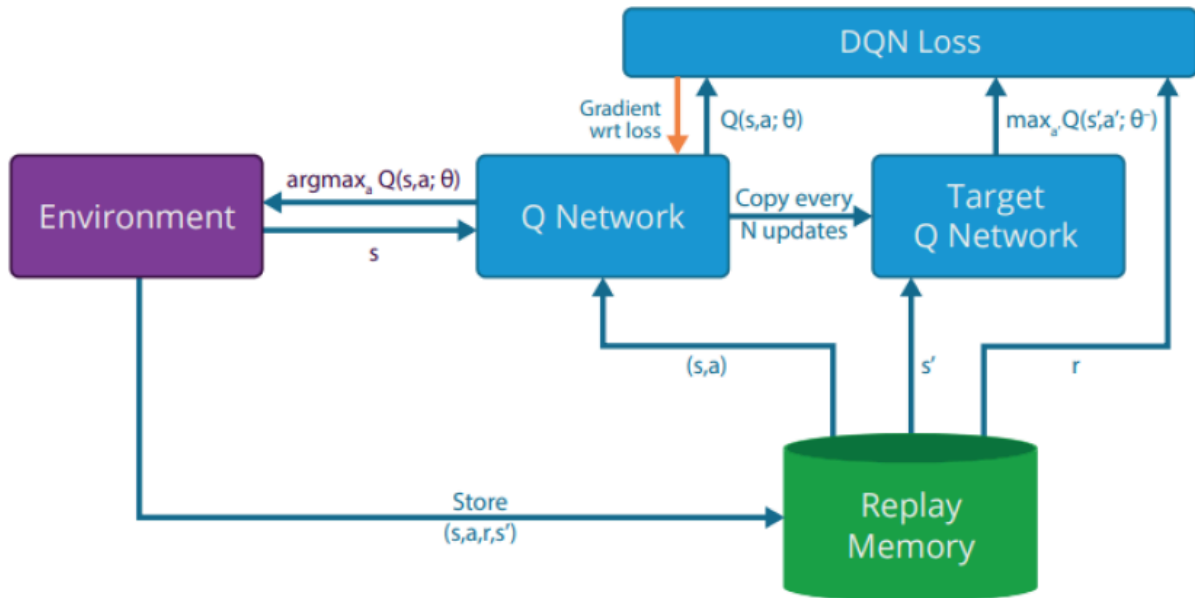
→ 딥러닝 기반 Q-learning은 타겟값도 네트워크에 의존하기 때문에, 네트워크가 계속 바뀌면 **타겟이 불안정하게 변동**

→ 타겟 네트워크를 일정 시간 동안 고정

→ 일정 주기마다 Q-Network의 가중치를 복사해서 Target-Network를 업데이트

14. DQN 아키텍처

▶ DQN architecture (Deepmind, 2015)



1. Environment (환경)

- 에이전트는 현재 상태 s 를 환경으로부터 받아옵니다.
- Q 네트워크는 상태 s 를 입력으로 받아 최적의 행동 a 를 선택합니다.
→ $\text{argmax}_a Q(s, a; \theta)$

2. Q Network (온라인 네트워크)

- 입력: 현재 상태 s
- 출력: 각 행동에 대한 Q값
- 목적: 현재 상태에서 어떤 행동을 취했을 때 얼마나 좋은지 평가
- 이 네트워크의 파라미터는 학습 대상입니다.

3. Target Q Network (타깃 네트워크)

- Q 네트워크와 동일한 구조이지만, 일정 주기마다 Q 네트워크의 가중치를 복사합니다.
- 예측한 Q값의 타깃을 안정적으로 생성하기 위해 존재

4. Replay Memory (경험 재현 메모리)

- 에이전트의 경험 (s, a, r, s') 를 저장
- 학습 시 랜덤 샘플링하여 데이터의 상관관계 제거
- DQN의 핵심 기법 중 하나

5. DQN Loss 계산

- 예측값: $Q(s, a; \theta)$
- 타깃값: $r + \gamma \max_{a'} Q(s', a'; \theta^-)$
- 손실 함수:

$$\mathcal{L} = \left(Q(s, a; \theta) - \left[r + \gamma \max_{a'} Q(s', a'; \theta^-) \right] \right)^2$$

- 이 손실을 기반으로 Q 네트워크를 학습
- 전체 동작 흐름 요약
 1. 상태 s 관측
 2. Q 네트워크로 행동 a 선택 ($\text{argmax } Q(s, a; \theta)$)
 3. 환경에 행동을 전달하고, 보상 r 과 다음 상태 s' 획득
 4. (s, a, r, s') 를 리플레이 메모리에 저장
 5. 리플레이 메모리에서 무작위 샘플로 Q 네트워크 업데이트
 6. 일정 주기로 타깃 네트워크를 Q 네트워크의 파라미터로 복사
 - 타깃 분리(Target Network) → 학습 안정화
 - 경험 재현(Replay Memory) → 상관 제거
 - 미니배치 학습 + 손실 함수 최적화 → 일반화 가능성 향상

► DQN (Deep Q learning, 2015)

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$

For episode = 1, M do

Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

For $t = 1, T$ do

With probability ϵ select a random action a_t

otherwise select $a_t = \arg\max_a Q(\phi(s_t), a; \theta)$

Execute action a_t in emulator and observe reward r_t and image x_{t+1}

Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$

Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ with respect to the network parameters θ

Every C steps reset $\hat{Q} = Q$

End For

End For

• Experience Replay

• Fixed Q-Target

• Experience Replay (녹색 박스)

- D : 경험을 저장하는 버퍼 (replay memory)
- 한 번의 경험을 학습에 여러 번 사용할 수 있게 해줌
- 상관관계를 줄이고 데이터 분포를 더 안정적으로 만듦

Initialize replay memory D to capacity N

Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

Sample minibatch from D

• Fixed Q-Target (주황색 박스)

- Q-타겟 추정에 사용하는 네트워크 $\hat{Q}$ 는 일정 주기로만 업데이트
- 학습 안정성을 높이는 중요한 기법

• 알고리즘 전체 흐름 정리

1. 초기화:

- Q-network Q 는 무작위 가중치로 시작
- Target Q-network $\hat{Q}$ 는 Q 의 복사본

2. 각 에피소드마다:

- 상태 s_1 을 초기화하고 전처리

3. 각 타임스텝 t 마다:

- ϵ -greedy 전략으로 행동 a_t 선택
- 환경에서 행동 수행 후, $(\phi_t, a_t, r_t, \phi_{t+1})$ 을 저장
- 리플레이 메모리에서 무작위 미니배치 샘플링
- 타깃 y_j 계산 (종료 여부 고려)
- **Q-network** 파라미터 θ 업데이트
→ 손실:

$$(y_j - Q(\phi_j, a_j; \theta))^2$$

- C 스텝마다 $\hat{Q} \leftarrow Q$ 로 동기화

• 요약

구성 요소	설명
Replay Memory	샘플 간 상관 줄이기, 효율적 학습
Target Network	Q-타깃의 안정성 확보
ϵ -Greedy	탐색과 활용의 균형 유지
Gradient Descent	MSE 기반 Q 업데이트